

Trabalho Prático de Dependência

ADS306 - Desenvolvimento para Dispositivos Móveis e Games

Prof. João Vítor Rodrigues de Vasconcelos

joaovasconcelos@univicosa.com.br

1. Objetivo

O presente trabalho de dependência tem como objetivo central avaliar a capacidade do discente em recuperar e aplicar os conceitos fundamentais abordados ao longo do semestre na disciplina de Desenvolvimento para Dispositivos Móveis e Games. A atividade consiste no planejamento e desenvolvimento de um “App Híbrido: Utilitário + Game 2D” responsivo, construído utilizando o framework React Native e o ecossistema Expo Go.

2. Requisitos Técnicos

O desenvolvimento do aplicativo deverá ser dividido em três fases distintas, contemplando a ementa completa da disciplina:

Fase 1: Fundamentos de UI e Estado

- **Arquitetura e Componentização:** O aplicativo deve ser modularizado. Crie componentes reutilizáveis para botões, cabeçalhos e cartões informativos.
- **Estilização:** A interface deve ser integralmente construída utilizando **Flexbox** para garantir a responsividade em diferentes tamanhos de tela.
- **Gerenciamento de Estado:** Uso obrigatório dos *Hooks* `useState` e `useEffect` para controle de variáveis locais e ciclo de vida.

Fase 2: Navegação e Consumo de Dados

- **Roteamento:** Implementar o **React Navigation**, utilizando obrigatoriamente um sistema de abas (**Tab Navigator**) para alternar entre a área utilitária e o jogo, e um **Stack Navigator** para o detalhamento de itens.
- **Integração de API:** A área utilitária do aplicativo deve consumir uma API RESTful pública (como **PokeAPI** ou **OpenWeather**) via **Axios** ou **Fetch**, exibindo os dados em uma **FlatList**.

Fase 3: Mecânica de Jogo 2D

- **Física e Loop:** A aba do jogo deve conter um ambiente 2D funcional gerenciado por um *Game Loop* nativo utilizando `requestAnimationFrame`.
- **Mecânicas Básicas:** Implementar movimentação de pelo menos um *sprite* na tela, sistema de pontuação atualizado em tempo real e detecção de colisão simples (AABB) entre o jogador e elementos do cenário.

Exemplo de estrutura de código exigida na Fase 3:

```
import React, { useState, useEffect } from "react";
import { View, Text, StyleSheet } from "react-native";

export default function GameScreen() {
  const [score, setScore] = useState(0);

  useEffect(() => {
    let animationFrameId;
    const gameLoop = () => {
      // Logica de atualizacao e colisao aqui
      animationFrameId = requestAnimationFrame(gameLoop);
    };
    gameLoop();
    return () => cancelAnimationFrame(animationFrameId);
  }, []);

  return (
    <View style={styles.container}>
      <Text>Pontuacao: {score}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: "#fff" }
});
```

3. Entregáveis

O discente deverá submeter, via ambiente virtual de aprendizagem, os seguintes artefatos:

1. **Repositório do Código:** Link público do repositório no GitHub contendo todo o código-fonte do projeto, com histórico de *commits* que comprove o desenvolvimento gradual.
2. **Apresentação:** Marcar uma apresentação com o professor através do e-mail, que será feita utilizando o Google Meet, e o aluno terá no máximo 10 minutos e depois deverá responder perguntas a respeito do trabalho.

4. Critérios de Avaliação

A nota final do trabalho prático (0 a 100 pontos) será distribuída da seguinte forma:

- **Interface e Experiência do Usuário (20%):** Qualidade visual, uso correto de Flexbox e componentização.
- **Lógica e Estrutura de Código (20%):** Uso adequado de Hooks, modularização e boas práticas (Clean Code).
- **Integração e Roteamento (30%):** Implementação fluida do React Navigation e consumo correto e resiliente (tratamento de erros) da API externa.
- **Mecânica do Jogo (30%):** Funcionamento adequado do *Game Loop*, colisões registradas corretamente e reatividade dos comandos na tela.